

Module 16: Python DB and Framework

1. HTML in Python

Q1. Introduction to embedding HTML within Python using web frameworks like Django or Flask

Embedding HTML within Python can be done using the web framework Django. Django is a high-level Python framework used to develop dynamic and database-driven web applications.

In Django, HTML is not written directly inside Python code. Instead, HTML files are stored in a **templates folder**, and Python handles backend logic through views. These views send data to templates, and the Django template engine renders dynamic HTML pages.

Django follows the **MVT (Model-View-Template)** architecture:

- **Model:** Handles database
- **View:** Contains business logic
- **Template:** Displays HTML

Steps to Embed HTML in Django

1. Create a Django project and app
2. Register the app in **settings.py**
3. Create view function in **views.py**
4. Configure URL in **urls.py**
5. Create HTML file in templates folder

App Registration in settings.py

After creating an app, it must be registered in Django settings.

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'myapp', # register your app here  
]
```

Example: Embedding HTML using Django

views.py

```
from django.shortcuts import render

def home(request):
    data = {
        'name': 'Rahul'
    }
    return render(request, 'index.html', data)
```

urls.py

```
from django.contrib import admin
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home),
]
```

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Django Example</title>
</head>
<body>

<h1>Welcome to Django</h1>
<p>Hello {{ name }}</p>

</body>
</html>
```

Explanation:

In this example, the app is first registered in **settings.py**. The **views.py** file sends data to the template. The **urls.py** file connects the URL with the view function. The **index.html** file displays dynamic data using `{{ name }}`.

Q2. Generating Dynamic HTML Content using Django Templates

Generating dynamic HTML content means creating web pages where the content changes based on user input or data. In Django, this is achieved using the Django Template Engine.

In Django, templates allow developers to insert dynamic data into HTML using special syntax. The data is sent from the view (Python code) to the template, and the template displays it dynamically.

Django templates help separate business logic from presentation, making the code clean and maintainable.

Django template features include:

- Variables using `{{ }}`
- Conditions using `{% if %}`
- Loops using `{% for %}`
- Template inheritance

Steps to Generate Dynamic HTML Content

1. Create a view function in `views.py`
2. Pass data from view to template
3. Create HTML file in templates folder
4. Use template syntax to display dynamic data

Example: Generating Dynamic HTML using Django

views.py

```
from django.shortcuts import render

def student(request):
    data = {
        'name': 'Rahul',
        'marks': 85,
        'subjects': ['Maths', 'Science', 'English']
    }
    return render(request, 'student.html', data)
```

urls.py

```
from django.contrib import admin
from django.urls import path
from . import views
```

```
urlpatterns = [
```

```
    path('student/', views.student),  
]
```

student.html

```
<!DOCTYPE html>  
<html>  
<head>  
    <title>Student Page</title>  
</head>  
<body>  
  
<h1>Student Details</h1>  
  
<p>Name: {{ name }}</p>  
<p>Marks: {{ marks }}</p>  
  
{% if marks > 50 %}  
    <p>Status: Pass</p>  
{% else %}  
    <p>Status: Fail</p>  
{% endif %}  
  
<h2>Subjects:</h2>  
<ul>  
{% for sub in subjects %}  
    <li>{{ sub }}</li>  
{% endfor %}  
</ul>  
  
</body>  
</html>
```

Explanation:

In this example, the **views.py** file sends data (name, marks, subjects) to the template. The **urls.py** file connects the URL with the view function. The **student.html** file uses template syntax like {{ }}, {% if %}, and {% for %} to display dynamic content.

2. CSS in Python

1. Integrating CSS with Django Templates

CSS (Cascading Style Sheets) is used to design and style HTML pages. In Django, CSS is integrated using **static files system**.

Django allows static files (CSS, JavaScript, images) to be stored either:

- Inside the **app-level static folder**, or
- Inside the **project-level static folder (recommended for common files)**

This helps in separating frontend design from backend logic.

Static Folder Location

✓ Project level static folder (Recommended):

```
project/  
|  
|— static/  
|   └─ style.css
```

✓ App level static folder:

```
app/  
|— static/  
|   └─ app_name/  
|       └─ style.css
```

settings.py Changes

```
STATIC_URL = 'static/'
```

```
STATICFILES_DIRS = [  
    BASE_DIR / "static",  
]
```

Steps to Integrate CSS

1. Create static folder (project or app level)
2. Add CSS file inside static folder

3. Configure settings.py
 4. Load static in HTML using {% load static %}
 5. Link CSS using {% static %}
-

Example

style.css

```
body{
    background-color: lightblue;
}

h1 {
    color: darkblue;
}
```

index.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>Django CSS</title>
    <link rel="stylesheet" href="{% static 'style.css' %}">
</head>
<body>

<h1>Welcome to Django</h1>

</body>
</html>
```

Explanation:

In Django, static files can be stored in app level or project level folders. Project level static folder is commonly used for global files. The **settings.py** file is configured to access static files, and templates load CSS using {% static %}.

Q2. How to Serve Static Files (CSS, JavaScript) in Django

Static files like CSS, JavaScript, and images are used to design and add functionality to web pages. In Django, these files are managed using a system called **static files handling**.

Django serves static files by storing them in a **static folder** and configuring settings in the project so that templates can access them easily.

Steps to Serve Static Files in Django

1. Create a **static folder** in the project (or app)
 2. Store CSS, JavaScript, and image files inside it
 3. Configure **settings.py** for static files
 4. Load static files in HTML templates
 5. Use {% static %} tag to access files
-

Project Structure Example

```
project/
|
|— static/
|   |— style.css
|   |— script.js
|— templates/
|   |— index.html
```

settings.py Configuration

```
STATIC_URL = 'static/'
```

```
STATICFILES_DIRS = [
    BASE_DIR / "static",
]
```

Example: Using CSS and JavaScript

style.css

```
body {
    background-color: lightyellow;
}
```

```
h1 {
```

```
color: green;
}
```

script.js

```
alert("Welcome to Django Static Files");
```

index.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>Static Files Example</title>

  <link rel="stylesheet" href="{% static 'style.css' %}">
</head>
<body>

<h1>Django Static Files Demo</h1>

<script src="{% static 'script.js' %}"></script>

</body>
</html>
```

Explanation:

In Django, static files are stored in a static folder. The **settings.py** file is configured using `STATIC_URL` and `STATICFILES_DIRS`. In templates, `{% load static %}` is used to enable static file loading, and `{% static %}` is used to link CSS and JavaScript files.

3. JavaScript with Python

Q1. Using JavaScript for Client-Side Interactivity in Django Templates

JavaScript is used in Django templates to make web pages interactive without reloading the page. It runs in the user's browser and improves user experience.

In Django templates, JavaScript can be written directly inside HTML or used from external files.

Example of Inline JavaScript in Django Template

```
{% load static %}
<!DOCTYPE html>
```

```
<html>
<head>
  <title>JavaScript Example</title>
</head>
<body>

<h1>Welcome to Django</h1>

<button onclick="showMessage()">Click Me</button>

<script>
function showMessage() {
  alert("Hello! This is JavaScript in Django template");
}
</script>

</body>
</html>
```

Explanation:

- JavaScript runs in the browser (client-side).
- It adds interactivity like button click, alerts, form validation, etc.
- In Django templates, JavaScript can be directly written inside `<script>` tags.

Q2. Linking External or Internal JavaScript Files in Django

In Django, JavaScript files are usually stored in the **static folder** and linked in HTML templates using Django's static system.

Steps:

1. Create a static folder
 2. Add JavaScript file inside it
 3. Configure settings.py
 4. Load static in template
 5. Link JavaScript using `{% static %}`
-

Project Structure

```
project/
|
|-- static/
|   |-- script.js
|-- templates/
|   |-- index.html
```

settings.py Configuration

```
STATIC_URL = 'static/'
```

```
STATICFILES_DIRS = [
    BASE_DIR / "static",
]
```

External JavaScript File (script.js)

```
function showMessage() {
    alert("Hello from External JavaScript File");
}
```

index.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>External JS Example</title>
</head>
<body>

<h1>Django JavaScript Demo</h1>

<button onclick="showMessage()">Click Me</button>

<script src="{% static 'script.js' %}"></script>

</body>
</html>
```

Explanation:

- External JavaScript files are stored in the static folder.
- Django loads them using {% load static %}.
- {% static 'script.js' %} is used to link the file.
- This makes code more organized and reusable.

4. Django Introduction

1 Overview of Django (Web Development Framework)

Django is designed to simplify the development of complex web applications. It follows the principle of “Don’t Repeat Yourself (DRY)” and helps developers write clean and reusable code. It includes many built-in tools, so developers do not need to write everything from scratch.

Django is widely used for:

- Web applications
- E-commerce websites
- Content management systems
- Social media platforms

Advantages of Django (e.g., scalability, security).

1. High Security

Django provides built-in protection against common security threats like SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).

2. Scalability

Django is highly scalable and can handle large amounts of traffic and data efficiently, making it suitable for big applications.

3. Fast Development

Because of its built-in features and reusable components, development is faster compared to many other frameworks.

4. Built-in Admin Panel

Django provides an automatic admin interface for managing application data easily.

5. Strong Community Support

It has a large community and extensive documentation, making it easier to learn and troubleshoot.

3 Django vs Flask Comparison

Flask is another popular Python web framework, but it is lightweight and minimal compared to Django.

1. Structure

- Django: Full-stack framework with built-in features
- Flask: Micro-framework with minimal structure

2. Flexibility

- Django: Less flexible but more standardized
- Flask: Highly flexible and customizable

3. Learning Curve

- Django: Slightly complex due to many built-in features
- Flask: Easier to learn for beginners

4. Use Case

- Django: Large-scale and complex applications
- Flask: Small projects, APIs, and lightweight applications

Which to Choose and Why

- Choose **Django** when building large, secure, and scalable web applications because it provides built-in features and strong security.
- Choose **Flask** when you need a simple, lightweight, and highly customizable application.

5. Virtual Environment

1 Understanding the Importance of a Virtual Environment in Python Projects

A virtual environment in Python is an isolated environment that allows each project to have its own dependencies, libraries, and Python versions without interfering with other projects or the system-wide Python installation.

In Python development, using a virtual environment is very important for maintaining clean, organized, and conflict-free projects.

venv is commonly used to create such isolated environments.

Importance of Virtual Environment

1. Dependency Management

Different Python projects may require different versions of libraries. A virtual environment allows each project to manage its own dependencies independently.

2. Avoid Version Conflicts

Without a virtual environment, installing packages globally can cause version conflicts between multiple projects. Virtual environments prevent this issue.

3. Clean System Environment

It keeps the global Python installation clean by installing all project-specific packages inside the virtual environment only.

4. Easy Project Setup and Sharing

A virtual environment makes it easier to share projects with others because all required dependencies can be listed and recreated easily.

5. Better Development Practice

Using virtual environments is considered a professional and standard practice in Python development to ensure stability and consistency.

2. Using venv or virtualenv to Create Isolated Environments

In Python development, isolated environments are created to keep project dependencies separate. This helps each project run with its own required libraries without affecting other projects. This can be done using tools like **venv** and **virtualenv**.

venv is a built-in Python module used to create virtual environments, while virtualenv is an external tool used for the same purpose, especially useful in older setups and advanced configurations.

Steps to Create Virtual Environment using venv

1. Create Virtual Environment

A virtual environment is created using the command:

- `python -m venv env_name`

This creates a new folder with an isolated Python environment.

2. Activate Virtual Environment

After creation, it must be activated:

- **Windows:** `env_name\Scripts\activate`
- **Mac/Linux:** `source env_name/bin/activate`

Once activated, all installed packages will stay inside this environment only.

3. Install Required Packages

After activation, packages can be installed using:

- `pip install package_name`

These packages will not affect the global Python installation.

4. Deactivate Environment

To exit the virtual environment:

- `deactivate`
-

Using virtualenv

- First install: `pip install virtualenv`
- Create environment: `virtualenv env_name`
- Activate and use it similar to `venv`

6. Project and App Creation

1 Steps to create a Django project and individual apps within the project.

In web development using Django, a project is the main container for a web application, and apps are smaller modules inside the project that handle specific functionalities like login, blog, or product management.

Django follows a modular structure where one project can contain multiple apps.

Steps to Create a Django Project

1. Install Django

First install Django using pip:

- `pip install django`
-

2. Create a Project

Create a new Django project using the command:

- `django-admin startproject project_name`

This creates the main project folder with necessary configuration files.

3. Navigate to Project Folder

- `cd project_name`
-

Steps to Create an App inside the Project

4. Create an App

Inside the project, create an app using:

- `python manage.py startapp app_name`

This creates a separate module for a specific feature.

5. Register the App

The newly created app must be added in **settings.py** inside `INSTALLED_APPS`.

6. Run the Server

Start the development server using:

- `python manage.py runserver`

2 Understanding the role of manage.py, urls.py, and views.py.

In web development using Django, different files are responsible for different tasks in the application. Among them, **manage.py**, **urls.py**, and **views.py** are the most important files in a Django project.

1. Role of manage.py

manage.py is a command-line utility used to manage the Django project. It helps to run various administrative tasks easily.

Main Functions:

- Run the development server (runserver)
- Create apps (startapp)
- Apply migrations (migrate)
- Create database tables
- Manage project settings indirectly

👉 It acts as a control file for the entire Django project.

2. Role of urls.py

urls.py is used for URL routing in Django. It connects URLs with specific views (functions).

Main Functions:

- Maps URL paths to views
- Handles user requests based on URL
- Helps in navigation between pages

👉 It works like a “road map” that directs requests to the correct function.

3. Role of views.py

views.py contains the main business logic of the application.

Main Functions:

- Processes user requests
- Retrieves and manipulates data
- Returns responses (HTML pages or data)
- Acts as a bridge between Model and Template

👉 It is the core logic handler of the Django application.

7. MVT Pattern Architecture

1 Django's MVT (Model-View-Template) architecture and how it handles request-response cycles.

In web development using Django, the framework follows the **MVT (Model-View-Template)** architecture. It is a design pattern used to separate an application into three main components so that development becomes clean, structured, and easy to maintain.

1. MVT Architecture

Model

The Model is responsible for managing the database and data structure. It defines how data is stored and retrieved.

View

The View contains the business logic of the application. It processes user requests, interacts with the Model if needed, and sends data to the Template.

Template

The Template is responsible for the presentation layer. It displays data in HTML format to the user.

2. Request-Response Cycle in Django

The request-response cycle explains how Django handles user actions step by step:

Step 1: User Request

A user enters a URL in the browser.

Step 2: URL Routing

The request first goes to the URL configuration file (**urls.py**), which maps the URL to a specific view.

Step 3: View Processing

The request is passed to the View, which processes the logic and may interact with the database through the Model.

Step 4: Template Rendering

The View sends data to the Template, which converts it into an HTML page.

Step 5: Response Sent

Finally, the rendered HTML page is sent back to the user as a response.

3. Working Flow Summary

User → URL → View → Model (optional) → Template → Response

8. Django Admin Panel

1 Introduction to Django's Built-in Admin Panel

In web development using Django, the **admin panel** is a built-in feature that provides an easy interface to manage application data without writing extra code.

The Django admin panel is automatically generated and allows developers or administrators to perform CRUD operations (Create, Read, Update, Delete) on database models through a web interface.

What is Django Admin Panel?

The Django admin panel is a ready-made web interface provided by Django to manage website data. It is mainly used for:

- Managing users
 - Adding or deleting records
 - Updating database information
 - Monitoring application data
-

Features of Django Admin Panel

1. Auto-generated Interface

Django automatically creates an admin interface based on defined models.

2. Easy Data Management

It allows easy insertion, updating, and deletion of database records.

3. User Authentication

Only authorized users (like superusers) can access the admin panel.

4. Secure Access

Django admin panel is protected with login authentication.

5. Customizable Interface

Developers can customize the admin panel according to project requirements.

How Django Admin Panel Works

1. Models are defined in the application
 2. These models are registered in the admin interface
 3. Django automatically creates a web-based dashboard
 4. Admin users log in and manage data easily
-

Advantages of Django Admin Panel

- Saves development time
- No need to create separate admin UI
- Easy database management
- Secure and reliable system
- Helpful for testing and development

2 Customizing the Django Admin Interface to Manage Database Records

In web development using Django, the **admin interface** is a built-in feature that allows developers to manage database records easily. Django also provides the ability to **customize the admin panel** so that data can be displayed and managed in a more organized and user-friendly way.

Customizing the admin interface helps in improving usability, readability, and efficiency while handling large amounts of data.

Why Customize Django Admin?

Customization of the admin panel is done to:

- Improve data visibility
 - Make record management easier
 - Add filters and search options
 - Display important fields only
 - Enhance user experience for admin users
-

How Django Admin Works

Django admin works with models. When a model is registered in the admin panel, Django automatically creates a default interface to manage its records.

However, this default interface can be customized using admin classes.

Ways to Customize Django Admin Interface

1. List Display

It allows selecting which fields should be shown in the admin list view.

2. Search Functionality

Adds a search bar to find records quickly.

3. Filtering Records

Allows filtering data based on specific fields.

4. Ordering

Controls how records are displayed (ascending or descending).

5. Editable Fields

Some fields can be made editable directly from the list view.

Example of Customization (Conceptual)

- Show only important fields like name, email
 - Add search by name
 - Filter records by category or date
 - Arrange records in proper order
-

Advantages of Custom Django Admin

- Better data management
- Saves time for administrators
- Easy navigation of records
- Improves productivity
- Makes large datasets easier to handle

9. URL Patterns and Template Integration

1 Setting up URL Patterns in `urls.py` for Routing Requests to Views

In web development using Django, URL routing is the process of connecting web addresses (URLs) to specific functions in the application. This is done using the **`urls.py`** file.

The main purpose of URL patterns is to direct user requests to the correct view so that the appropriate response is generated.

What is `urls.py`?

`urls.py` is a configuration file in Django used to define URL patterns. It acts as a map that links URLs to view functions.

When a user enters a URL in the browser, Django checks `urls.py` to decide which view should handle the request.

How URL Routing Works in Django

1. User enters a URL in the browser
 2. Django checks the **`urls.py`** file
 3. URL pattern is matched
 4. Request is sent to the corresponding view
 5. View returns a response (HTML or data)
-

Steps to Set Up URL Patterns

1. Import Required Modules

Import path and views module in urls.py.

2. Define URL Patterns

Create a list called urlpatterns to define routes.

3. Map URL to View

Connect each URL to its corresponding view function.

Types of URL Patterns

1. Simple URL Pattern

Used for basic page routing like home, about, etc.

2. Dynamic URL Pattern

Used to pass parameters in URL (like id, name, etc.)

Advantages of URL Routing

- Organizes web application structure
- Makes navigation easy
- Connects user requests with backend logic
- Supports dynamic content handling
- Improves readability and maintenance

2 Integrating Templates with Views to Render Dynamic HTML Content

In web development using Django, templates and views work together to generate dynamic web pages. This integration allows Python data to be displayed inside HTML pages in a structured way.

Templates handle the presentation (HTML), while views handle the logic and data processing. When both are combined, Django can render dynamic content based on user requests.

What are Views and Templates?

Views

views.py contains the business logic of the application. It processes user requests, prepares data, and sends it to templates.

Templates

Templates are HTML files that display data dynamically using Django template syntax like `{{ }}` and `{% %}`.

How Views and Templates Work Together

1. User sends a request (URL)
 2. Django routes request to a view
 3. View processes data and sends it to template
 4. Template displays data in HTML format
 5. Final response is sent to the browser
-

Steps to Integrate Views with Templates

1. Create View Function

View function prepares data and selects template.

2. Pass Data to Template

Data is sent from view to template using context.

3. Create HTML Template

Template displays dynamic data using variables.

4. Render Template

View renders HTML page and sends response to user.

Example of Dynamic Content Flow

- View sends: name = Rahul
 - Template displays: Hello Rahul
-

Advantages of Integration

- Enables dynamic web pages
- Separates logic and presentation
- Makes code reusable and clean
- Easy to manage large applications

- Improves development efficiency

10. Form Validation using JavaScript

Using JavaScript for Front-End Form Validation

In web development using Django, form validation is used to check user input before sending it to the server. **JavaScript** is used for client-side (front-end) validation to give instant feedback to the user and improve user experience.

JavaScript helps in checking required fields, email format, password length, and other input rules before form submission.

How Form Validation Works

1. User fills the form
2. JavaScript checks input values
3. If error → shows message
4. If correct → form is submitted

Example: Form Validation using JavaScript

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Form Validation</title>
```

```
<script>
```

```
function validateForm() {
```

```
    let name = document.forms["myForm"]["name"].value;
```

```
    let email = document.forms["myForm"]["email"].value;
```

```
    let password = document.forms["myForm"]["password"].value;
```

```
    if (name == "") {
```

```
    alert("Name is required");  
    return false;  
}
```

```
if (email == "") {  
    alert("Email is required");  
    return false;  
}
```

```
if (password.length < 6) {  
    alert("Password must be at least 6 characters");  
    return false;  
}
```

```
    alert("Form submitted successfully!");  
    return true;  
}  
</script>
```

</head>

<body>

<h2>Registration Form</h2>

<form name="myForm" onsubmit="return validateForm()">

Name: <input type="text" name="name">

Email: <input type="text" name="email">

Password: <input type="password" name="password">

<input type="submit" value="Submit">

</form>

</body>

</html>

Explanation:

- The form calls validateForm() on submit
- JavaScript checks each field
- If any error is found, form submission is stopped using return false
- If all inputs are correct, form is submitted successfully

11. Django Database Connectivity (MySQL or SQLite)

1 Connecting Django to a Database (SQLite or MySQL)

In web development using Django, a database is used to store and manage application data such as users, products, and records. Django provides built-in support to connect with databases like **SQLite** and **MySQL**.

By default, Django uses SQLite, but it can also be configured to use MySQL or other databases.

1. Default Database (SQLite)

SQLite is the default database used in Django. It does not require any external setup and stores data in a single file.

SQLite Configuration in settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Advantages of SQLite

- No installation required
 - Easy to use
 - Good for small projects and testing
-

2. Connecting Django with MySQL

MySQL is a powerful database used for large-scale applications.

Steps to Connect MySQL

Step 1: Install MySQL driver

```
pip install mysqlclient
```

Step 2: Create Database in MySQL

Example:

```
CREATE DATABASE myproject;
```

Step 3: Configure settings.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'myproject',
        'USER': 'root',
        'PASSWORD': 'your_password',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

Step 4: Apply Migrations

```
python manage.py migrate
```

How Django Uses Database

1. Models define structure of data
 2. Django converts models into database tables
 3. Data is stored and retrieved using ORM
 4. Views interact with database through models
-

Advantages of Django Database Integration

- Easy configuration
- Supports multiple databases
- Built-in ORM (Object Relational Mapping)
- Secure and scalable
- Reduces SQL complexity

2 Using the Django ORM for Database Queries

In web development using Django, database operations are performed using a powerful feature called **ORM (Object Relational Mapper)**. Django ORM allows developers to interact with the database using Python code instead of writing raw SQL queries.

It automatically converts Python objects into database tables and SQL queries, making database handling easier and safer.

What is Django ORM?

Django ORM is a layer between Python code and the database. It allows developers to:

- Create database tables using Python classes (Models)
 - Insert, update, delete, and fetch data using Python methods
 - Avoid writing complex SQL queries
-

Common Django ORM Operations

1. Create (Insert Data)

Used to add new records into the database.

```
Student.objects.create(name="Rahul", marks=85)
```

2. Read (Fetch Data)

Get all records

```
Student.objects.all()
```

Get single record

```
Student.objects.get(id=1)
```

Filter data

```
Student.objects.filter(marks__gt=50)
```

3. Update Data

Used to modify existing records.

```
student = Student.objects.get(id=1)  
student.marks = 90  
student.save()
```

4. Delete Data

Used to remove records from the database.

```
student = Student.objects.get(id=1)  
student.delete()
```

Advantages of Django ORM

- No need to write SQL queries
- Easy to use with Python syntax
- Secure (reduces SQL injection risk)
- Database independent (works with SQLite, MySQL, etc.)
- Faster development process

How ORM Works in Django

1. Model is created in Python
2. Django converts model into database table
3. ORM translates Python queries into SQL
4. Database returns results to Django

12. ORM and QuerySets

1 Understanding Django's ORM and How QuerySets Are Used to Interact with the Database

In web development using Django, database operations are handled using ORM (Object Relational Mapping). Django ORM allows developers to interact with the database using Python code instead of writing SQL queries.

It provides an abstraction layer between Python and the database, making database operations simple, secure, and efficient.

What is Django ORM?

Django ORM converts Python classes (models) into database tables and Python queries into SQL queries automatically.

It helps developers to:

- Create and manage database tables using models
- Perform CRUD operations easily
- Avoid writing complex SQL queries

What is a QuerySet?

A **QuerySet** is a collection of database queries in Django ORM. It represents a set of data retrieved from the database.

QuerySet is used to:

- Retrieve data from database

- Filter data based on conditions
 - Order and manipulate data
 - Perform database operations efficiently
-

Key Features of QuerySets

1. Lazy Execution

QuerySets are not executed immediately. They run only when data is actually needed.

2. Chainable

Multiple filters and conditions can be combined together.

3. Efficient

Reduces unnecessary database calls and improves performance.

Common QuerySet Methods

1. all()

Returns all records from a table.

```
Student.objects.all()
```

2. filter()

Returns records matching a condition.

```
Student.objects.filter(marks__gt=50)
```

3. get()

Returns a single record.

```
Student.objects.get(id=1)
```

4. exclude()

Returns records excluding a condition.

```
Student.objects.exclude(marks__lt=40)
```

5. `order_by()`

Sorts the data.

```
Student.objects.order_by('name')
```

How ORM and QuerySets Work Together

1. Developer writes Python query using ORM
 2. ORM converts it into SQL
 3. QuerySet is created as result
 4. Database returns data
 5. Django displays result in application
-

Advantages of Using ORM and QuerySets

- No need for SQL writing
- Easy and readable code
- Secure against SQL injection
- Faster development
- Works with multiple databases

13. Django Forms and Authentication

1. Using Django's Built-in Form Handling

Definition

Django Forms are used to take input from users, validate the data, and process it easily.

Django provides built-in classes for form handling.

Why Use Django Forms?

- Easy to create forms
- Automatic validation

- Secure handling of user data
 - Reduces manual coding
 - Generates HTML forms automatically
-

Types of Forms in Django

1. Form Class (forms.Form)

Used when form is not directly connected to database.

Example

```
# forms.py
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=50)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
```

2. ModelForm (forms.ModelForm)

Used when form is connected to database model.

Example

```
# models.py
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=50)
    email = models.EmailField()

# forms.py
from django.forms import ModelForm
from .models import Student

class StudentForm(ModelForm):
    class Meta:
        model = Student
        fields = '__all__'
```

Steps of Form Handling

Step 1: Create Form

```
# forms.py
from django import forms

class UserForm(forms.Form):
    username = forms.CharField(max_length=50)
    password = forms.CharField(widget=forms.PasswordInput)
```

Step 2: Create View

```
# views.py
from django.shortcuts import render
from .forms import UserForm

def register(request):

    if request.method == 'POST':
        form = UserForm(request.POST)

        if form.is_valid():
            print(form.cleaned_data)
        else:
            form = UserForm()

    return render(request, 'register.html', {'form': form})
```

Step 3: Create Template

```
<!-- register.html -->

<form method="POST">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Submit</button>
</form>
```

Important Methods

Method	Meaning
is_valid()	Checks form validation
cleaned_data	Returns validated data
save()	Saves ModelForm data

Advantages of Django Forms

- Automatic validation
 - Better security
 - Easy error handling
 - Reusable code
 - CSRF protection
-

2. Implementing Django's authentication system (sign up, login, logout, password management).

Definition

Django provides a built-in authentication system for:

- User Registration (Sign Up)
- Login
- Logout
- Password Change
- Password Reset

Django uses the built-in User model.

Authentication Features

Feature	Work
Sign Up	Create new user
Login	User authentication

Feature	Work
Logout	End user session
Password Change	Change current password
Password Reset	Reset forgotten password

Import Authentication Modules

```
from django.contrib.auth.models import User
from django.contrib.auth import authenticate, login, logout
```

1. User Registration (Sign Up)

View

```
# views.py
from django.contrib.auth.models import User
from django.shortcuts import render, redirect

def signup(request):

    if request.method == 'POST':
        username = request.POST['username']
        password = request.POST['password']

        User.objects.create_user(
            username=username,
            password=password
        )

        return redirect('login')

    return render(request, 'signup.html')
```

Template

```
<form method="POST">
    {% csrf_token %}

    <input type="text" name="username" placeholder="Username">
```

```
<input type="password" name="password" placeholder="Password">

<button type="submit">Signup</button>
</form>
```

2. Login

View

```
from django.contrib.auth import authenticate, login

def user_login(request):

    if request.method == 'POST':

        username = request.POST['username']
        password = request.POST['password']

        user = authenticate(
            request,
            username=username,
            password=password
        )

        if user is not None:
            login(request, user)
            return redirect('home')

    return render(request, 'login.html')
```

3. Logout

View

```
from django.contrib.auth import logout

def user_logout(request):
    logout(request)
    return redirect('login')
```

4. Password Change

Django provides built-in views for password change.

urls.py

```
from django.urls import path
from django.contrib.auth import views as auth_views
```

```
urlpatterns = [

    path(
        'password_change/',
        auth_views.PasswordChangeView.as_view(),
        name='password_change'
    ),

]
```

5. Password Reset

Used when user forgets password.

Django sends reset link through email.

urls.py

```
path(
    'password_reset/',
    auth_views.PasswordResetView.as_view(),
    name='password_reset'
),
```

Authentication Functions

Function	Meaning
----------	---------

authenticate()	Verify username & password
----------------	----------------------------

login()	Login user
---------	------------

logout()	Logout user
----------	-------------

create_user()	Create new user
---------------	-----------------

Advantages of Django Authentication

- Built-in security
- Password hashing

- Session management
- Easy user handling
- Ready-made authentication system

14. CRUD Operations using AJAX

Definition of AJAX

AJAX stands for:

Asynchronous JavaScript And XML

AJAX is a technique used to send and receive data from the server **without refreshing or reloading the webpage**.

What is Asynchronous Request?

Asynchronous means:

The browser can send request to server and continue working without waiting for response.

This makes the webpage faster and smoother.

Why AJAX is Used?

- No full page reload
 - Faster communication
 - Better user experience
 - Dynamic content update
 - Reduces server load
-

How AJAX Works

1. User performs an action (click, submit, etc.)
2. JavaScript sends request to server
3. Server processes request
4. Server sends response back

5. Webpage updates dynamically without reload

AJAX Flow Diagram

User Action



JavaScript AJAX Request



Server Processing



Response from Server



Update Webpage Without Reload

Basic AJAX Syntax

```
$.ajax({  
  
    url: "data.php",  
  
    method: "POST",  
  
    data: {  
        name: "Bharti"  
    },  
  
    success: function(response){  
        console.log(response);  
    }  
});
```

Explanation of AJAX Properties

Property Meaning

url Server file location

method Type of request (GET/POST)

data Data sent to server

success Function executed after response

Example of AJAX Request

HTML

```
<input type="text" id="name">

<button onclick="sendData()">Send</button>

<div id="result"></div>
```

JavaScript AJAX

```
function sendData(){

    var username = $("#name").val();

    $.ajax({

        url: "server.php",

        method: "POST",

        data: {
            name: username
        },

        success: function(response){

            $("#result").html(response);

        }

    });

}
```

Advantages of AJAX

Advantage	Description
Faster Website	Only required data loads

Advantage	Description
No Page Reload	Smooth user experience
Interactive UI	Dynamic content update
Saves Bandwidth	Smaller data transfer

Disadvantages of AJAX

Disadvantage	Description
JavaScript Required	Browser must support JS
Debugging Difficult	Harder than normal requests
Security Issues	Requires validation

Common AJAX Methods

Method Use

GET	Retrieve data
POST	Send data
PUT	Update data
DELETE	Delete data

Real-Life Examples of AJAX

- Gmail inbox loading
- Facebook comments
- Instagram likes
- Live search suggestions
- Shopping cart updates

15. Customizing the Django Admin Panel

Definition

The Django Admin Panel is a built-in feature of Django used to manage database data through a web interface.

Django allows developers to customize the admin panel according to project requirements.

Why Customize Django Admin?

- Better user interface
 - Easy data management
 - Display important fields
 - Add search and filters
 - Improve admin productivity
-

Basic Admin Registration

models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=50)
    email = models.EmailField()
    city = models.CharField(max_length=30)

    def __str__(self):
        return self.name
```

admin.py

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

Techniques for Customizing Django Admin

1. Using ModelAdmin Class

ModelAdmin is used to customize how models appear in admin panel.

Example

```
from django.contrib import admin
from .models import Student

class StudentAdmin(admin.ModelAdmin):
    pass

admin.site.register(Student, StudentAdmin)
```

2. list_display

Used to show fields in admin table view.

Example

```
class StudentAdmin(admin.ModelAdmin):

    list_display = ['id', 'name', 'email', 'city']
```

3. list_filter

Adds filter option in sidebar.

Example

```
class StudentAdmin(admin.ModelAdmin):

    list_filter = ['city']
```

4. search_fields

Used to search records in admin panel.

Example

```
class StudentAdmin(admin.ModelAdmin):

    search_fields = ['name', 'email']
```

5. list_per_page

Controls number of records per page.

Example

```
class StudentAdmin(admin.ModelAdmin):  
  
    list_per_page = 10
```

6. ordering

Used to sort records.

Example

```
class StudentAdmin(admin.ModelAdmin):  
  
    ordering = ['name']
```

7. readonly_fields

Makes fields read-only.

Example

```
class StudentAdmin(admin.ModelAdmin):  
  
    readonly_fields = ['id']
```

8. fieldsets

Groups fields into sections.

Example

```
class StudentAdmin(admin.ModelAdmin):  
  
    fieldsets = (  
  
        ('Personal Info', {  
            'fields': ('name', 'email')  
        }),  
  
        ('Location Info', {
```

```
        'fields': ('city',)
    }),

)
```

9. Customizing Admin Site Title

admin.py

```
admin.site.site_header = "My Admin Panel"
```

```
admin.site.site_title = "Admin Portal"
```

```
admin.site.index_title = "Welcome to Dashboard"
```

10. Inline Models

Used to edit related models on same page.

Example

```
class CourseInline(admin.TabularInline):
    model = Course
```

Complete Example

```
from django.contrib import admin
from .models import Student
```

```
class StudentAdmin(admin.ModelAdmin):
```

```
    list_display = ['id', 'name', 'email', 'city']
```

```
    search_fields = ['name']
```

```
    list_filter = ['city']
```

```
    ordering = ['name']
```

```
    list_per_page = 5
```

```
admin.site.register(Student, StudentAdmin)
```

Advantages of Admin Customization

Advantage	Description
Easy Data Management	Manage records quickly
Better UI	Improved admin interface
Fast Searching	Easy search option
Organized Data	Filters and sorting available

16. Payment Integration Using Paytm

1. Introduction to Integrating Payment Gateways (Like Paytm) in Django Projects

Definition

A Payment Gateway is a service that allows online applications to accept digital payments securely.

In Django projects, payment gateways like:

- Paytm
- Razorpay
- Stripe
- PayPal

can be integrated to process online payments.

Why Use Payment Gateway?

- Accept online payments
 - Secure transactions
 - Fast payment processing
 - Supports cards, UPI, wallets, net banking
 - Useful for e-commerce and booking systems
-

How Payment Gateway Works

Customer → Website → Payment Gateway → Bank

↓

Payment Success/Failure

Steps to Integrate Paytm in Django

1. Create Paytm Merchant Account

First create merchant account on:

[Paytm Business](#)

You will get:

- Merchant ID
 - Merchant Key
 - Website Name
-

2. Install Required Package

```
pip install paytmchecksum
```

3. Add Paytm Credentials

settings.py

```
PAYTM_MERCHANT_ID = 'Your_Merchant_ID'
```

```
PAYTM_MERCHANT_KEY = 'Your_Merchant_Key'
```

```
PAYTM_WEBSITE = 'WEBSTAGING'
```

```
PAYTM_CALLBACK_URL = 'http://127.0.0.1:8000/callback/'
```

4. Create Payment View

views.py

```
from django.shortcuts import render
from django.http import HttpResponse
import paytmchecksum
```

```

def payment(request):

    param_dict = {

        "MID": "Your_Merchant_ID",

        "ORDER_ID": "ORDER0001",

        "TXN_AMOUNT": "100",

        "CUST_ID": "CUST001",

        "INDUSTRY_TYPE_ID": "Retail",

        "WEBSITE": "WEBSTAGING",

        "CHANNEL_ID": "WEB",

        "CALLBACK_URL": "http://127.0.0.1:8000/callback/",

    }

    checksum = paytmchecksum.generateSignature(
        param_dict,
        "Your_Merchant_Key"
    )

    param_dict['CHECKSUMHASH'] = checksum

    return render(request, 'paytm.html', {'param_dict': param_dict})

```

5. Create Payment Template

paytm.html

```

<form method="post"
action="https://securegw-stage.paytm.in/order/process/">

{% for key, value in param_dict.items %}

<input type="hidden"
name="{{key}}"
value="{{value}}">

```

```
{% endfor %}
```

```
</form>
```

```
<script>
```

```
    document.forms[0].submit();
```

```
</script>
```

6. Handle Payment Response

views.py

```
from django.views.decorators.csrf import csrf_exempt
```

```
@csrf_exempt
```

```
def callback(request):
```

```
    if request.method == "POST":
```

```
        response = request.POST
```

```
        return HttpResponse("Payment Successful")
```

Important Features of Payment Gateway

Feature	Description
Secure Payment	Encrypted transactions
Payment Verification	Confirms payment status
Multiple Payment Options	UPI, cards, wallets
Callback Response	Returns transaction result

Common Payment Gateway Terms

Term	Meaning
Merchant ID	Unique business ID
Checksum	Security verification
Callback URL	URL after payment

Term	Meaning
Transaction ID	Unique payment ID

Advantages of Payment Gateway Integration

- Online payment support
 - Secure transactions
 - Easy order management
 - Better customer experience
 - Automated payment processing
-

Real-Life Examples

Payment gateways are used in:

- Food ordering websites
- Shopping websites
- Ticket booking systems
- Online course platforms

17. GitHub Project Deployment

Steps to push a Django project to GitHub.

1.Open project folder in VS Code

2.Open Terminal

3.Initialize git

`git init`

4.Add all files

`git add .`

5.Commit files

`git commit -m "First commit"`

6.Create repository on GitHub

7.Copy repository URL

8.Connect local project with GitHub

git remote add origin REPOSITORY_URL

9.Check branch name

git branch

10.Rename branch to main

git branch -M main

11.Push project to GitHub

git push -u origin main

12.Enter GitHub username and password/token if asked

18. Live Project Deployment (PythonAnywhere)

Introduction to Deploying Django Projects to Live Servers like PythonAnywhere

Definition

Deployment means:

Uploading a Django project from local computer to a live server so users can access it online through the internet.

Example:

Local Project → Live Server → Public Website

What is PythonAnywhere?

[PythonAnywhere Official Website](#)

PythonAnywhere is a cloud hosting platform specially designed for Python and Django applications.

It allows developers to deploy Django websites easily.

Why Use PythonAnywhere?

- Easy Django deployment
- Free hosting available
- No server setup required

- Supports MySQL and SQLite
 - Accessible from anywhere
-

Basic Requirements Before Deployment

- Django project ready
 - GitHub account (optional)
 - PythonAnywhere account
 - requirements.txt file
 - Virtual environment
-

Main Steps for Django Deployment

Step Work

- 1 Upload Django project
 - 2 Create virtual environment
 - 3 Install Django
 - 4 Configure WSGI file
 - 5 Set static files
 - 6 Reload web app
-

Important Deployment Settings

settings.py

DEBUG = False

ALLOWED_HOSTS = ['yourusername.pythonanywhere.com']

Install Requirements

pip freeze > requirements.txt

Upload Project

Project can be uploaded using:

- GitHub
 - File upload
 - Git clone
-

Create Virtual Environment

```
mkvirtualenv myenv --python=python3.10
```

Install Django on Server

```
pip install django
```

Configure WSGI File

WSGI file connects Django project with server.

Example:

```
import os
from django.core.wsgi import get_wsgi_application
```

```
os.environ.setdefault(
    'DJANGO_SETTINGS_MODULE',
    'projectname.settings'
)
```

```
application = get_wsgi_application()
```

Apply Migrations

```
python manage.py migrate
```

Collect Static Files

```
python manage.py collectstatic
```

Reload Web App

After configuration:

Click “Reload” button in PythonAnywhere dashboard.

Advantages of Deploying Django Projects

Advantage	Description
Live Website	Accessible online
Real User Access	Anyone can use website
Professional Hosting	Better project presentation
Cloud Storage	Secure hosting